A PROJECT REPORT ON

HIGH THROUGHPUT WRITE-OPTIMIZED KEY-VALUE STORE USING LSM TREE ENGINE

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (COMPUTER ENGINEERING)

SUBMITTED BY

Chaitanya Paraskar	Exam No: B400050361
Shreyash Dhas	Exam No: B400050389
Shreyash Gajbhiye	Exam No: B400050407
Shreyas Gopnarayan	Exam No: B400050418

DEPARTMENT OF COMPUTER ENGINEERING
PUNE INSTITUTE OF COMPUTER TECHNOLOGY
DHANKAWADI, PUNE 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2025-2026

CERTIFICATE

This is to certify that the project report entitled

High Throughput Write-Optimised Key-Value Store using LSM Tree Engine

Submitted by

Chaitanya Paraskar	Exam No: B400050361
Shreyash Dhas	Exam No: B400050389
Shreyash Gajbhiye	Exam No: B400050407
Shreyas Gopnarayan	Exam No: B400050418

are bonafide students of this institute and the work has been carried out by them under the supervision of **Prof. M. V. Mane** and it is approved for the partial fulfillment of the requirement of **Savitribai Phule Pune University**, for the award of the degree of Bachelor of Engineering (Computer Engineering).

Prof. M. V. Mane
Internal Guide
Dept. of Computer Engg.

Dr. B. A. Sonkamble
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place: Pune
Date:

ACKNOWLEDGEMENT

It gives us great pleasure in presenting the project report on **High Throughput Write-Optimised Key-Value Store using LSM Tree Engine**.

We would like to take this opportunity to thank our guide **Prof. M. V. Mane** for giving us all the help and guidance we needed. We are really grateful for her kind support and valuable suggestions throughout the development of this project.

We are also grateful to **Dr. B. A. Sonkamble**, Head of Computer Engineering Department, PICT, for his indispensable support and suggestions.

We express our sincere gratitude to the Principal, **Dr. S. T. Gandhe**, for providing us with the necessary infrastructure and resources. We also thank all faculty members and laboratory assistants of the Computer Engineering Department for their cooperation in the completion of this project.

Chaitanya Paraskar
Shreyash Dhas
Shreyash Gajbhiye
Shreyas Gopnarayan
(B.E. Computer Engineering)

ABSTRACT

Traditional database storage engines built around B-trees are optimized for read-heavy workloads but suffer under intensive write operations due to random disk I/O. The Log-Structured Merge Tree (LSM tree) addresses this by converting random writes into sequential ones — buffering data in memory and periodically flushing sorted batches to disk. This architecture underpins widely adopted systems such as LevelDB, RocksDB, and Cassandra.

This report presents KiwiDB, a prototype write-optimized key-value store built on the LSM tree architecture. The system implements the complete data lifecycle of an LSM-based engine: write-ahead logging for crash recovery, an in-memory memtable backed by a concurrent skip list, on-disk sorted string tables (SSTables) with bloom filters and sparse indexes, a parallel flush pipeline for memory-to-disk persistence, and a multi-level compaction engine for space reclamation and read performance maintenance. A hybrid compaction strategy — tiering at Level 0 and leveling at subsequent levels — reduces write amplification while keeping read costs bounded. The prototype also includes a web-based dashboard for real-time inspection of engine state across all stages of the data lifecycle.

The single-node storage engine presented here serves as a foundational building block for distributed key-value systems. Production systems such as etcd, TiKV, and CockroachDB are built by layering consensus protocols (Raft, Paxos) and distributed transaction logic on top of a local LSM-based storage engine. KiwiDB's modular architecture — with well-defined interfaces between the WAL, memtable, SSTable, and compaction subsystems — is designed to allow such extensions, providing a base upon which replication, sharding, and distributed coordination can be composed.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	**Illustration**
LSM	Log-Structured Merge Tree
KV	Key-Value
WAL	Write-Ahead Log
SSTable	Sorted String Table
GIL	Global Interpreter Lock
API	Application Programming Interface
REST	Representational State Transfer
MVCC	Multi-Version Concurrency Control
CRC	Cyclic Redundancy Check
FPR	False Positive Rate
LRU	Least Recently Used
SPA	Single Page Application
FIFO	First In, First Out
ASGI	Asynchronous Server Gateway Interface
ABC	Abstract Base Class
DFD	Data Flow Diagram
UML	Unified Modeling Language
SDLC	Software Development Life Cycle
mmap	Memory-Mapped I/O

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

With the rapid growth of web applications and cloud computing, key-value (KV) stores have become a critical component of modern data infrastructure. A KV store maps a set of keys to associated values and can be considered a simplified interface over more complex storage systems. Without having to provide features usually required for a relational database system, KV stores offer higher performance, better scalability, and greater availability [1]. They play a central role in data centers supporting Internet-scale services, including BigTable [2] at Google, Cassandra [3] at Facebook, and Dynamo [4] at Amazon.

The B+ tree is a common structure used in traditional databases because its high fanout reduces the number of I/O operations for a query. However, it is highly inefficient for random insertions and updates. When there are intensive mutations, B+ trees lead to a large number of costly disk seeks and significantly degraded performance. The Log-Structured Merge Tree (LSM tree) [5] addresses this by transforming random writes into sequential writes: incoming data is sorted in an in-memory buffer and flushed to storage as a whole when the buffer is full. Many popular KV stores adopt this design, including BigTable [2], Cassandra [3], HBase [6], and LevelDB [7].

This project presents **KiwiDB**, a complete LSM tree key-value store implemented from scratch in Python 3.12+. The system comprises approximately 6,600 lines of type-checked Python code across 25 modules, with 211 automated tests, an async-native API, and a built-in web dashboard for real-time engine inspection.

1.2 MOTIVATION

Despite the maturity of existing LSM-based systems, we observe two limitations that motivate this work:

Synchronous APIs. LevelDB and RocksDB [8] expose blocking C++ interfaces. Modern Python applications built with frameworks like FastAPI and aiohttp run on `asyncio` event loops. Integrating a synchronous storage engine requires thread-pool wrappers that add latency, complexity, and concurrency bugs. To the best of our knowledge, no existing LSM implementation offers a natively asynchronous API.

Opaque Internals. Production engines prioritize throughput over observability. Inspecting memtable state, bloom filter hit rates, compaction progress, or SSTable layouts requires external tooling. Internal design decisions are documented sparsely, making these systems difficult to learn from.

Additionally, from an academic perspective, building a storage engine from

first principles provides deep insight into database internals — write-ahead logging, memory management, disk persistence, concurrency control, and background job scheduling — topics that are typically studied only theoretically.

1.3 PROBLEM DEFINITION AND OBJECTIVES

Problem Statement: Design and implement a write-optimized key-value store that provides an async-native API, crash-safe persistence, background compaction with GIL bypass, and full observability via a web dashboard.

Objectives:

1. Implement an **async-native API** where all public operations (`put`, `get`, `delete`, `flush`, `close`) are async coroutines that never block the event loop.
2. Implement a **write-ahead log** with CRC32 integrity verification for crash recovery.
3. Design a **concurrent skip list** memtable with fine-grained per-node locking for writes and lock-free reads.
4. Implement **SSTable persistence** with bloom filters, sparse indexes, and memory-mapped reads.
5. Design a **parallel flush pipeline** with event-chain ordered commits for concurrent SSTable writes.
6. Implement **subprocess-based compaction** via `ProcessPoolExecutor` to bypass CPython's GIL for true CPU parallelism.
7. Build a **web dashboard** with FastAPI backend and React frontend for real-time engine inspection.

1.4 PROJECT SCOPE AND LIMITATIONS

Scope: KiwiDB is a single-node, embedded key-value store supporting point reads and writes with crash safety. It includes a REPL interface, REST API, and web dashboard. It is designed as both a functional storage engine and an educational tool.

Limitations:

- No range query support (although the internal merge iterator could be extended).
- No block compression (LZ4/zstd).

- No multi-key transactions or batch writes.
- Single-node only; no replication or distributed operation.
- 10–100× lower raw throughput compared to C++ implementations due to Python's interpreted nature.

1.5 METHODOLOGIES OF PROBLEM SOLVING

The project follows an **iterative and incremental** development methodology. Each subsystem (WAL, memtable, SSTable, compaction, flush pipeline) was designed, implemented, and tested independently before being integrated into the engine coordinator. Key methodological practices include:

- **Test-driven development:** 211+ automated tests written alongside implementation using pytest and pytest-asyncio.
- **Formal type checking:** Dual static analysis with mypy (strict mode) and basedpyright (strict mode) to catch type errors at compile time.
- **Design from first principles:** Architecture informed by academic literature on LSM trees, with innovations tailored to Python's runtime model.
- **Security analysis:** bandit static security scanner run on every change.
- **Continuous documentation:** Design documents and API reference maintained alongside code using MkDocs.

CHAPTER 2

LITERATURE SURVEY

This chapter reviews the foundational literature and existing systems that inform the design of KiwiDB.

1. O'Neil et al. — The Log-Structured Merge Tree (1996) [5]

O'Neil et al. proposed the LSM tree as a write-optimized alternative to B-trees. The key insight is to buffer incoming writes in an in-memory component and periodically flush sorted runs to disk. On-disk components are merged in the background to reclaim space and maintain read performance. This design transforms random writes into sequential I/O, which is significantly faster on both HDDs and SSDs. The LSM tree forms the foundation of KiwiDB's architecture.

2. Google — LevelDB (2011) [7]

LevelDB is the canonical open-source LSM implementation, originating from Google's BigTable. It uses a skip list as the in-memory memtable, write-ahead logging for durability, and a multi-level compaction strategy where each level has non-overlapping key ranges. LevelDB serves as KiwiDB's primary baseline for comparison. However, LevelDB has two limitations: a single background thread for both flush and compaction, and a fully synchronous API. KiwiDB addresses both.

3. Facebook — RocksDB (2013) [8]

RocksDB extends LevelDB with concurrent compaction, column families, configurable compaction strategies, and a lock-free skip list using compare-and-swap (CAS) atomics. It remains the most widely deployed LSM engine. KiwiDB borrows the concept of multiple immutable memtables from RocksDB but adopts a different concurrency model suited to Python's GIL.

4. Wang et al. — LOCS: LSM on Open-Channel SSD (2014) [9]

Wang et al. extended LevelDB to exploit channel-level parallelism of open-channel SSDs by increasing the number of immutable memtables and flushing them concurrently to separate SSD channels. Their work demonstrated up to $4\times$ throughput improvement. KiwiDB shares the insight that concurrent flush improves throughput but achieves it at the software level with an event-chain commit mechanism on commodity storage.

5. Dayan et al. — Monkey: Optimal Navigable KV Store (2017) [10]

Monkey optimizes bloom filter memory allocation across LSM levels. By assigning more bits to lower levels (where reads are more frequent), Monkey minimizes the expected number of false positive I/Os. KiwiDB takes a different approach: instead

of cross-level optimization, each bloom filter is sized optimally using the *actual record count* at write time, producing right-sized filters without global tuning.

6. Dayan and Idreos — Dostoevsky (2018) [11]

Dostoevsky introduces "lazy leveling" as a hybrid between tiering and leveling. The largest level uses tiering (cheaper writes) while smaller levels use leveling (cheaper reads). KiwiDB's L0-tiering / L1-leveling strategy achieves a similar write amplification reduction through a simpler design: L0 files are unsorted (tiered), while L1+ levels maintain a single merged file each.

7. Luo and Carey — LSM-based Storage Techniques: A Survey (2020) [12]

This comprehensive survey formalizes the LSM design space across four dimensions: merge policy (tiering vs. leveling), size ratio, merge trigger, and component organization. It provides the theoretical framework within which KiwiDB's hybrid strategy can be analyzed. The survey also identifies write amplification as the primary bottleneck in leveled compaction — exactly what KiwiDB's L0 tiering addresses.

8. Bloom — Space/Time Trade-offs in Hash Coding (1970) [13]

Bloom proposed the probabilistic data structure now known as the bloom filter. It allows membership testing with no false negatives and a tunable false positive rate. Each SSTable in KiwiDB includes a bloom filter that eliminates unnecessary disk reads during point lookups. KiwiDB uses MurmurHash3 with seed variations as the hash function family.

9. Herlihy et al. — A Provably Correct Concurrent Skip List (2006) [14]

Herlihy et al. presented a concurrent skip list algorithm with fine-grained locking for writers and lock-free traversal for readers. KiwiDB's skip list implementation is adapted from this design, with per-node locks for writers and GIL-atomic boolean flag checks for lock-free readers — a simplification made possible by CPython's GIL guaranteeing atomic attribute reads.

10. Lakshman and Malik — Cassandra (2010) [3]

Cassandra is a distributed LSM-based storage system designed for high availability and partition tolerance. While KiwiDB is single-node, Cassandra's write path (commit log $\rightarrow$ memtable $\rightarrow$ SSTable) informed KiwiDB's architecture. Cassandra also demonstrates that LSM trees scale well in distributed environments — a potential future extension for KiwiDB.

CHAPTER 3

SOFTWARE REQUIREMENTS SPECIFICATION

3.1 ASSUMPTIONS AND DEPENDENCIES

The following assumptions and dependencies govern the design and operation of KiwiDB:

- Python 3.12 or higher is required (uses PEP 695 type alias syntax).
- The underlying filesystem supports `fsync()` for durable writes.
- The system operates on a single machine (no distributed coordination).
- Node.js 18+ is required for building the React web dashboard frontend.

Runtime Dependencies:

- `msgpack` — binary serialization for WAL entries
- `mmh3` — MurmurHash3 for bloom filter hash functions
- `cachetools` — LRU cache implementation for block cache
- `structlog` — structured logging framework
- `FastAPI` / `uvicorn` — REST API server
- `websockets` — live log streaming
- `prompt-toolkit` — interactive REPL

3.2 FUNCTIONAL REQUIREMENTS

3.2.1 FR-1: Async Put Operation

The system shall provide an asynchronous `put(key, value)` operation that atomically writes a key-value pair to the WAL and memtable. The WAL must be fsynced before the operation returns.

3.2.2 FR-2: Async Get Operation

The system shall provide an asynchronous `get(key)` operation that searches the active memtable, immutable queue, and SSTables (L0 through L3) in order, returning the value associated with the highest sequence number for the given key, or `None` if the key does not exist or has been deleted.

3.2.3 FR-3: Async Delete Operation

The system shall support key deletion through tombstone markers. A delete writes a special sentinel value that shadows the key during reads and is garbage-collected during compaction.

3.2.4 FR-4: Flush Operation

The system shall support both automatic and manual flush of the active memtable to an SSTable on disk. Automatic flush is triggered when the memtable exceeds a configurable threshold.

3.2.5 FR-5: Crash Recovery

On startup, the system shall replay the WAL to recover any writes that were not yet flushed to SSTables. Entries already persisted in SSTables (identified by sequence number) shall be skipped.

3.2.6 FR-6: Background Compaction

The system shall merge SSTables across levels (L0→L1→L2→L3) when size or count thresholds are exceeded. Compaction shall run in a subprocess to avoid blocking the main event loop.

3.2.7 FR-7: Web Dashboard

The system shall provide a web-based dashboard with pages for KV operations, memtable inspection, SSTable browsing, compaction monitoring, live log streaming, configuration management, and an interactive terminal.

3.2.8 FR-8: REPL Interface

The system shall provide an interactive command-line REPL supporting `put`, `get`, `del`, `flush`, `mem`, `disk`, `stats`, `config`, and `trace` commands.

3.2.9 FR-9: Runtime Configuration

The system shall support live-updatable configuration parameters (e.g., memtable size thresholds, bloom filter FPR, compaction thresholds) that take effect without restart.

3.2.10 FR-10: Live Log Streaming

The system shall broadcast structured log events via TCP and relay them to WebSocket clients for real-time monitoring.

3.3 EXTERNAL INTERFACE REQUIREMENTS

3.3.1 User Interfaces

- **REPL:** Interactive terminal interface built with prompt-toolkit.
- **Web Dashboard:** React SPA (Vite + TypeScript) with 10 pages.
- **REST API:** FastAPI server with 20+ endpoints for programmatic access.

3.3.2 Hardware Interfaces

The system interacts with the filesystem via standard POSIX system calls (`open`, `write`, `fsync`, `mmap`). NVMe storage is recommended for optimal parallel flush performance.

3.3.3 Software Interfaces

1. **Operating System:** Linux, macOS, or Windows with Python 3.12+
2. **Python asyncio:** Event loop for async operations
3. **ProcessPoolExecutor:** Subprocess boundary for compaction workers
4. **Node.js / npm:** Frontend build toolchain for React SPA

3.3.4 Communication Interfaces

- **HTTP/REST:** Port 8081 for API and web dashboard.
- **WebSocket:** `/ws/logs` endpoint for live log streaming.
- **TCP:** Internal log broadcast server for structured event relay.

3.4 NONFUNCTIONAL REQUIREMENTS

3.4.1 Performance Requirements

- Memtable insert: $O(\log n)$ via skip list.

- Point read: $O(\log n)$ memtable + $O(k)$ bloom checks per SSTable level.
- Lock-free reads: no write lock acquired during `get()` operations.

3.4.2 Safety Requirements

- WAL durability before visibility: `fsync()` completes before memtable update.
- CRC32 integrity verification on all WAL entries, bloom filters, and sparse indexes.
- SSTable `meta.json` written last as completeness signal.

3.4.3 Security Requirements

- Static security analysis via bandit on every code change.
- No external network dependencies in the core engine.
- No dynamic code execution or unsafe deserialization in the engine.

3.4.4 Software Quality Attributes

- **Maintainability:** 100% type annotation coverage with dual type checkers.
- **Testability:** Each subsystem independently testable via formal ABC contracts.
- **Observability:** Every critical path instrumented with structured logging.

3.5 SYSTEM REQUIREMENTS

3.5.1 Database Requirements

KiwiDB *is* the database. Data is stored as binary files (WAL log, SSTable data/index/filter files) and JSON manifests on the local filesystem. No external database is required.

3.5.2 Software Requirements

- Python 3.12+ (required for PEP 695 type syntax)
- Node.js 18+ (for building React frontend)
- `uv` package manager (recommended) or `pip`

3.5.3 Hardware Requirements

- Any modern x86_64 or ARM64 system with 2+ GB RAM.
- Storage: SSD recommended; NVMe optimal for parallel flush benefit.
- No GPU required.

3.6 ANALYSIS MODELS: SDLC MODEL

The project follows an **Iterative and Incremental** SDLC model. Each subsystem (WAL, memtable, SSTable, compaction, flush pipeline, web dashboard) was developed as an independent iteration with its own design, implementation, and testing cycle. Integration testing was performed after each iteration to verify subsystem interoperation.

This model was chosen because:

- Each subsystem has well-defined interfaces (formal ABCs and Protocols).
- Subsystems can be developed and tested independently.
- Risks (GIL blocking, crash safety, concurrency) are addressed incrementally.
- Feedback from testing each iteration informs the next.

CHAPTER 4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

KiwiDB is structured as a stack of cooperating layers. External clients enter through a thin HTTP / WebSocket presentation tier; a single `LSMEngine` coordinator delegates to five manager subsystems, each of which owns a set of low-level workers (skip list, WAL writer, SSTable reader/writer, bloom filter, block cache, merge iterator); persistence is handled by the local filesystem. Figure 4.1 shows this top-to-bottom view, and Figure 4.2 complements it with the strict module-level dependency hierarchy that enforces it.

4.1.1 System Overview

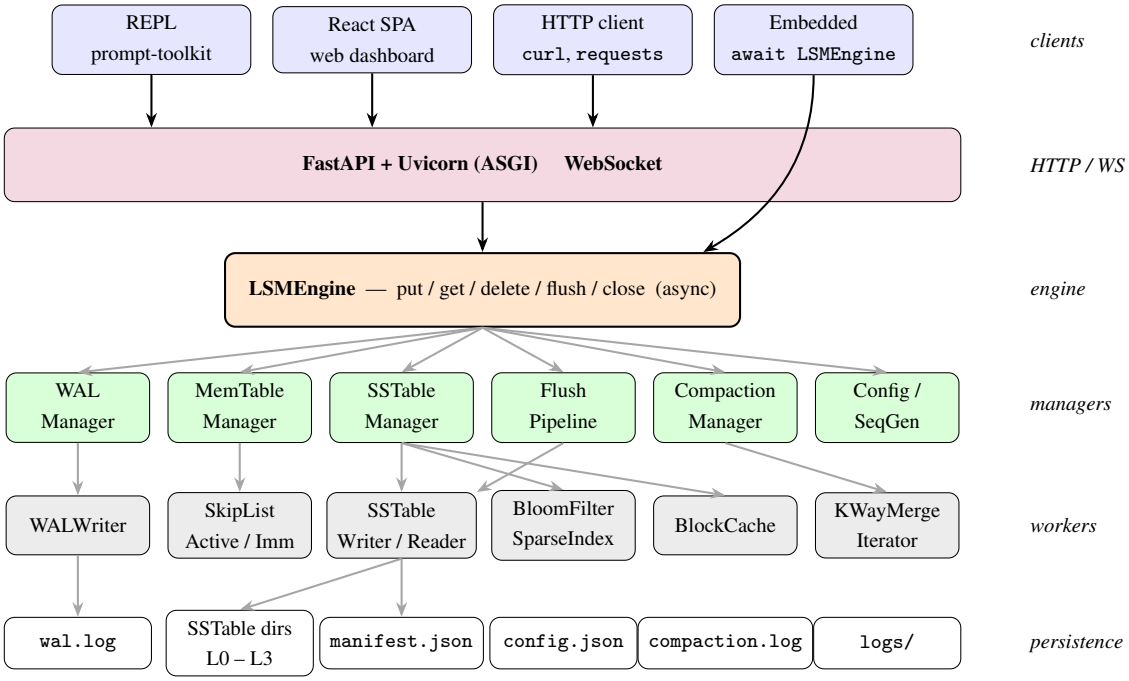

Figure 4.1: KiwiDB System Architecture.

- **Clients.** External consumers of the engine — the CLI REPL, the React web dashboard, arbitrary HTTP clients, and embedded Python applications that `await` the engine directly as a library.
- **Presentation (HTTP / WebSocket).** A FastAPI + Uvicorn ASGI process exposes the engine over eight REST routers and a WebSocket stream for live logs, translating network requests into async engine calls.
- **Engine (`LSMEngine`).** A thin async coordinator that owns the public API (`put`, `get`, `delete`, `flush`, `close`) and orchestrates sequence numbering, durability, memtable writes, and background work across its managers.

- **Managers.** Five subsystem managers — WAL, MemTable, SSTable, Flush Pipeline, Compaction — each own a single concern and hide their internal state behind a focused async interface consumed by the engine.
- **Workers.** Low-level data structures and file formats: the concurrent skip list, WAL writer, SSTable reader/writer, bloom filter and sparse index, block cache, and k-way merge iterator that perform the actual read, write, and merge work.
- **Persistence.** The local filesystem holds the durable state — `wal.log`, per-level SSTable directories (L0–L3), `manifest.json`, `config.json`, the compaction audit log, and structured log files.

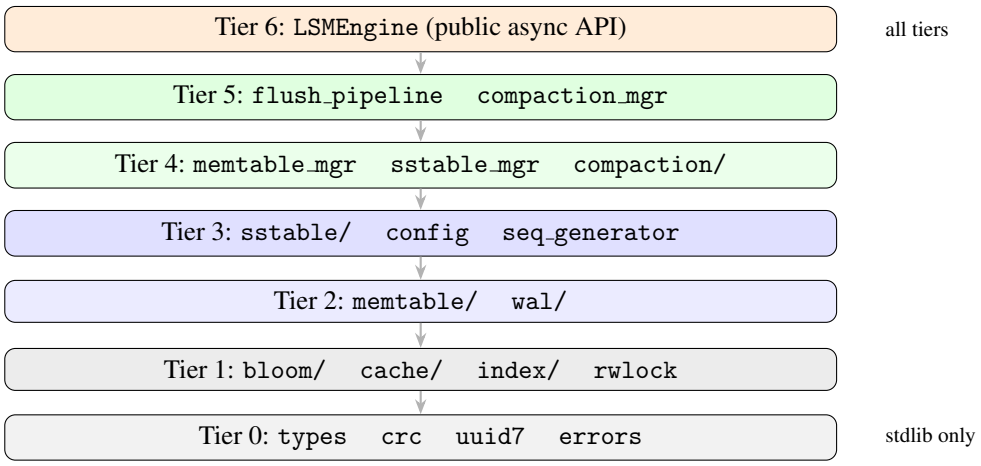

Figure 4.2: KiwiDB layered dependency model.

Component	**Model**	**Reason**
MemTable insert	Sync (threading locks)	Pure in-memory, microseconds
WAL append	Sync	Must fsync before returning
Flush pipeline	Async daemon task	Non-blocking I/O, parallel writes
Compaction merge	Subprocess (`ProcessPoolExecutor`)	CPU-bound, bypasses GIL
SSTable reads	Sync (mmap)	Memory-mapped, zero-copy
Config access	Sync (RLock)	Thread-safe attribute reads

Table 4.1: Thread and async model for each engine component.

4.2 MATHEMATICAL MODEL

4.2.1 State Space Formulation

Let K be the set of all possible keys and V the set of all possible values. The engine state is defined as a function $S : K \to (V \cup \{\perp\})$, where $\perp$ denotes a deleted or non-existent key. Each operation is associated with a monotonically increasing sequence number $\text{seq} \in \mathbb{N}$.

The three operations are:

$$\texttt{put}(k, v) : S \mapsto S[k \mapsto v] \text{ with } \text{seq} = \text{seq}_{\max} + 1 \tag{4.1}$$

$$\texttt{get}(k) : S \mapsto \begin{cases} v & \text{if } S(k) = v \\ \texttt{None} & \text{if } S(k) = \perp \end{cases} \tag{4.2}$$

$$\texttt{delete}(k) : S \mapsto S[k \mapsto \perp] \text{ with } \text{seq} = \text{seq}_{\max} + 1 \tag{4.3}$$

4.2.2 Bloom Filter Sizing

For a bloom filter with n elements and target false positive rate p, the optimal bit array size m and hash function count k are:

$$m = \left\lceil \frac{-n \cdot \ln p}{(\ln 2)^2} \right\rceil, \quad k = \left\lceil \frac{m}{n} \cdot \ln 2 \right\rceil \tag{4.4}$$

Unlike LevelDB (fixed 10 bits/key) and RocksDB (configurable bits/key), KiwiDB computes m and k from the *actual* record count at SSTable write time, producing optimally-sized filters for every SSTable.

4.2.3 Write Amplification Analysis

Let T be the size ratio (default 10), L the record size, and B the block size. Table **??** compares write amplification at flush time.

Strategy	**Flush Write Amp.**	**Read Amp. (bloom)**
LevelDB (pure leveling)	$O(T \times L/B)$	$L \times \phi = 0.04$
KiwiDB (hybrid)	$O(L/B)$	$T\phi + L\phi = 0.13$
Improvement	**10**$\times$	3.25$\times$ worse

Table 4.2: Write amplification comparison.

4.2.4 Skip List Complexity

The concurrent skip list uses MAX_LEVEL = 16 levels with promotion probability $p = 0.5$. Expected height for n elements is $E[\text{height}] = \log_{1/p}(n) = \log_2(n)$. All operations (insert, lookup, delete) have expected time complexity $O(\log n)$.

4.2.5 K-Way Merge Complexity

The compaction merge uses a min-heap of K sorted iterators over N total records. Time complexity is $O(N \log K)$ with $O(K)$ space for the heap.

4.3 DATA FLOW DIAGRAMS

4.3.1 DFD Level 0 — Context Diagram

Figure 4.3 shows the context-level data flow diagram.

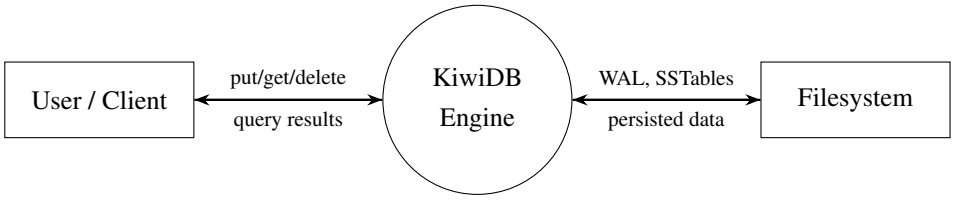

Figure 4.3: DFD Level 0 — Context Diagram.

4.3.2 DFD Level 1

Figure 4.4 expands the engine into its major subsystems.

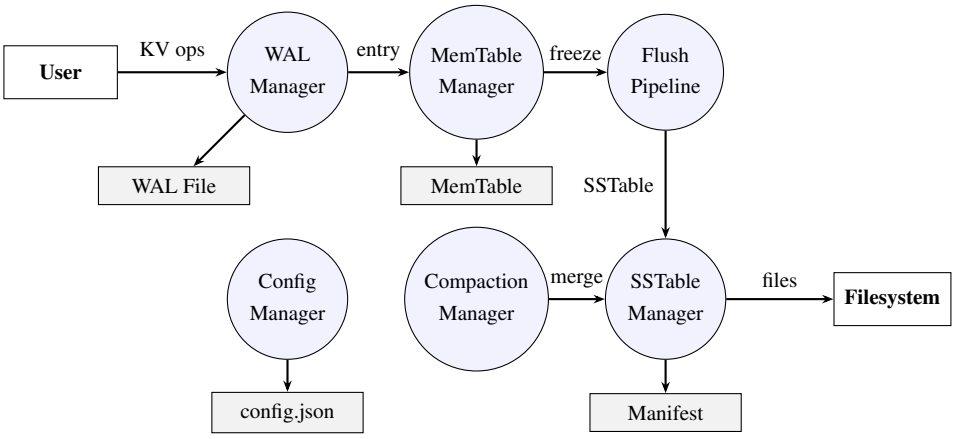

Figure 4.4: DFD Level 1 — Major subsystems and data stores.

4.3.3 DFD Level 2 — Write Path

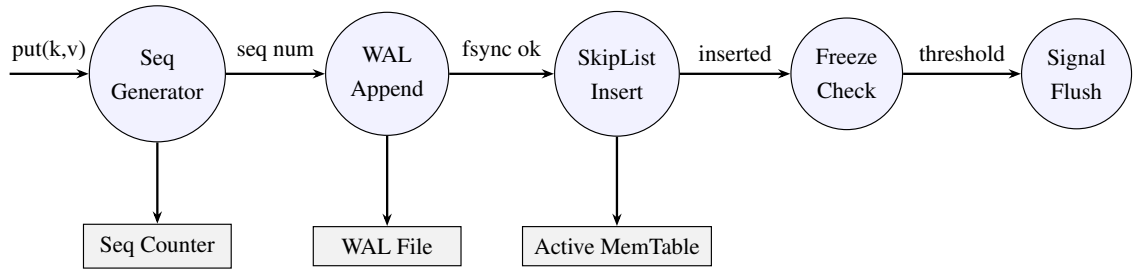

Figure 4.5: DFD Level 2 — Write path detail.

4.3.4 DFD Level 2 — Read Path

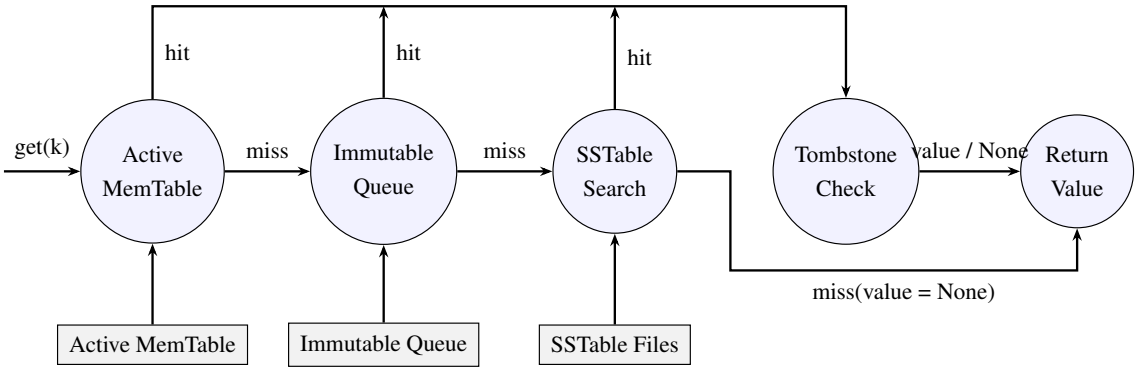

Figure 4.6: DFD Level 2 — Read path detail.

4.4 USE CASE DIAGRAM

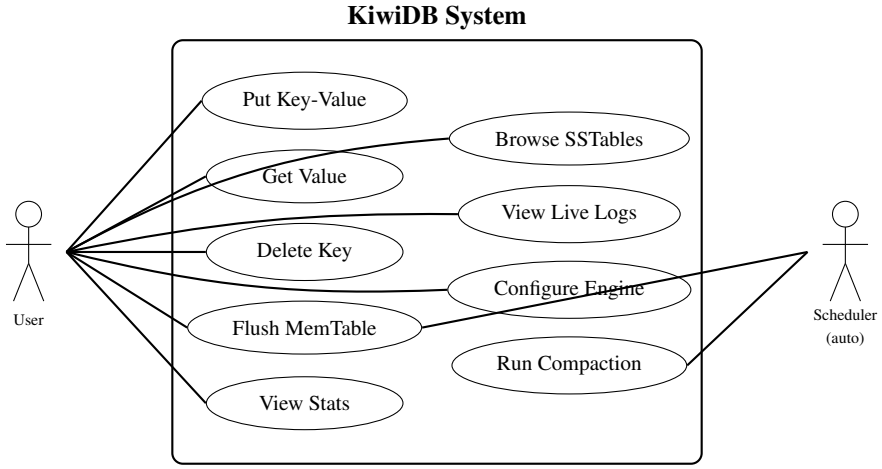

Figure 4.7: Use Case Diagram for KiwiDB.

4.5 UML DIAGRAMS

4.5.1 Activity Diagram — Write Path

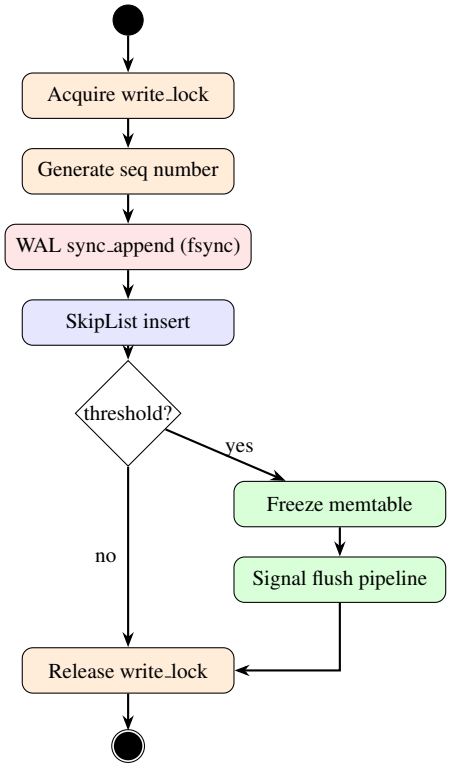

Figure 4.8: Activity Diagram — Write Path.

4.5.2 Activity Diagram — Read Path

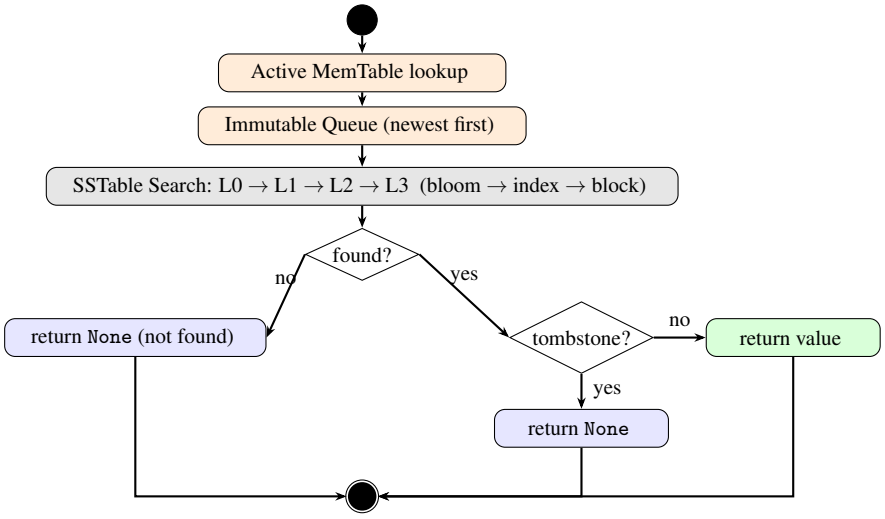

Figure 4.9: Activity Diagram — Read Path.

4.5.3 Sequence Diagram

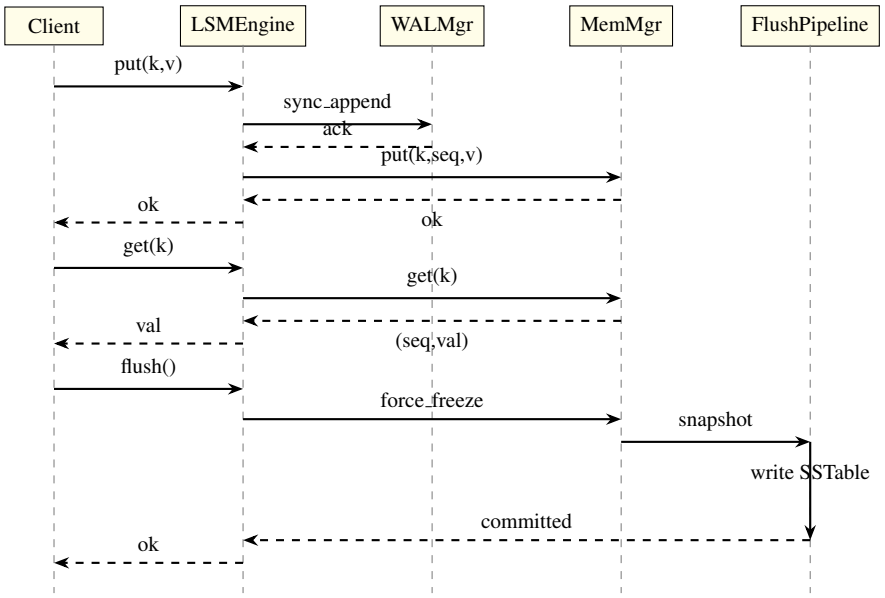

Figure 4.10: Sequence Diagram — Put, Get, and Flush operations.

4.5.4 Class Diagram

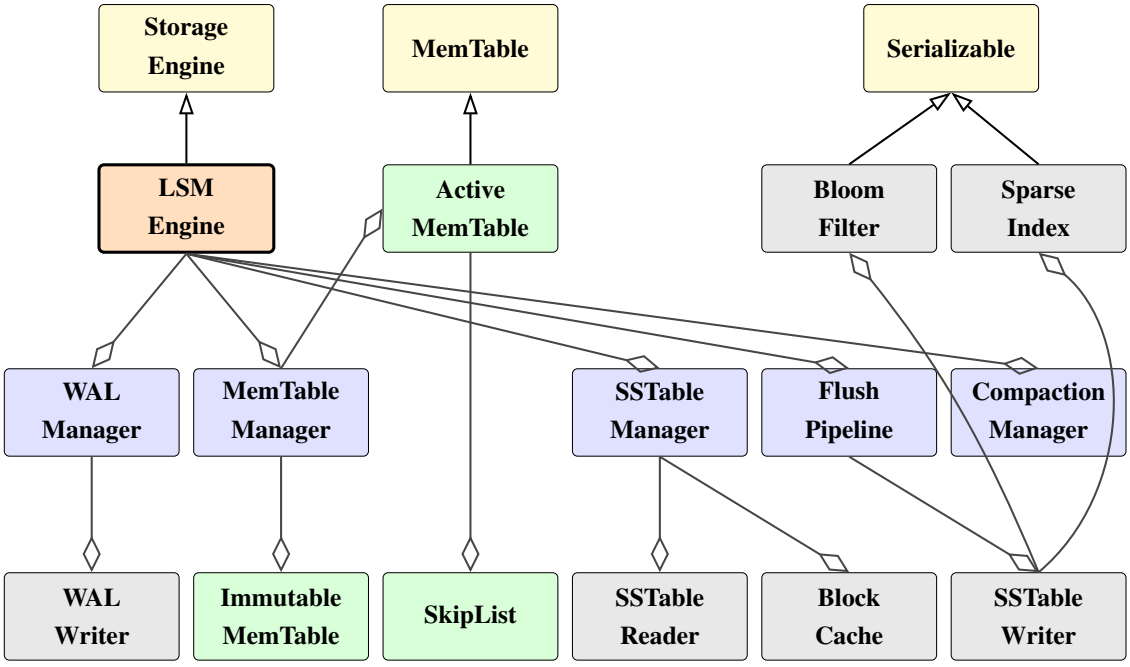

Figure 4.11: Class Diagram — Core class hierarchy.

4.5.5 Component Diagram

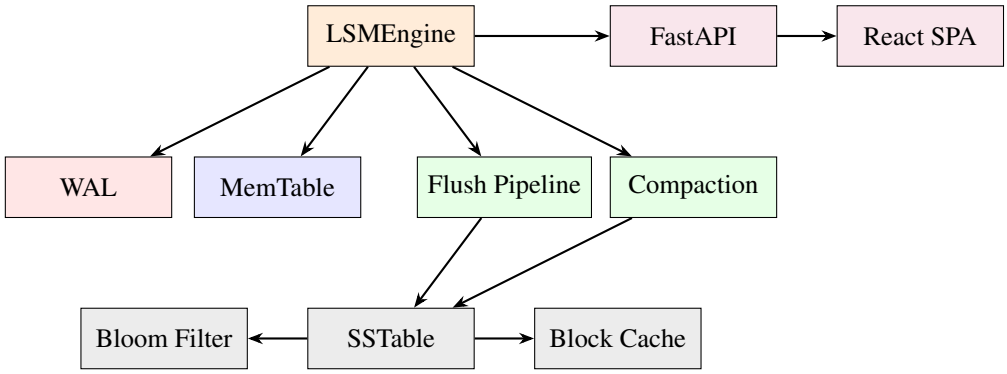

Figure 4.12: Component Diagram.

CHAPTER 5

PROJECT PLAN

5.1 PROJECT ESTIMATE

5.1.1 Reconciled Estimates

- Total lines of code: approximately 6,600 (core engine + web layer).
- Number of modules: 25 (12 core + 10 web + 3 CLI/tools).
- Number of test modules: 25 with 211+ individual test cases.
- Estimated development effort: 18 person-weeks across 4 team members.

5.1.2 Project Resources

- **Human Resources:** 4 developers (Chaitanya Paraskar, Shreyash Dhas, Shreyash Gajbhiye, Shreyas Gopnarayan) supervised by Prof. Madhuri Mane.
- **Hardware:** Development laptops with SSD storage.
- **Software:** Python 3.12+, VS Code, Git, GitHub, Node.js.

5.2 RISK MANAGEMENT

5.2.1 Risk Identification

Risk	**Prob.**	**Impact**	**Category**
GIL blocks compaction	High	High	Technical
Data corruption on crash	Medium	Critical	Safety
Memory growth (slow flush)	Medium	High	Performance
Concurrent read/write race	Medium	High	Correctness
Type safety regression	Low	Medium	Quality

Table 5.1: Identified project risks.

5.2.2 Risk Analysis

Each risk was analyzed for its root cause and potential impact:

- **GIL blocking:** CPython's GIL serializes CPU-bound threads. A compaction thread competing with writes degrades throughput.

- **Data corruption:** Power loss during write can leave partial data on disk.
- **Memory growth:** If flush is slower than write rate, the immutable queue grows unboundedly.
- **Race conditions:** Concurrent readers and writers on shared data structures can produce inconsistent results.

5.2.3 Overview of Risk Mitigation, Monitoring, Management

- **GIL bypass:** Compaction runs in `ProcessPoolExecutor` subprocess with its own GIL.
- **Crash safety:** WAL with CRC32 + fsync; `meta.json` written last as completeness signal.
- **Backpressure:** Immutable queue capped at 4 entries with 60-second timeout.
- **Concurrency safety:** Per-node skip list locks; GIL-atomic lock-free reads; AsyncRWLock for SSTable manager.
- **Type safety:** Dual type checkers (mypy + basedpyright) in strict mode.

5.3 PROJECT SCHEDULE

5.3.1 Project Task Set

1. Requirements analysis and architecture design
2. Core engine: WAL, MemTable (skip list), SSTable persistence
3. Flush pipeline with parallel writes and ordered commits
4. Compaction engine with subprocess workers
5. Block cache, bloom filter optimization, sparse index
6. Web dashboard: FastAPI backend + React frontend
7. Testing, documentation, and quality gates
8. Report writing and final presentation

5.3.2 Task Network

Tasks 1–2 are sequential foundations. Tasks 3–5 can be partially parallelized. Task 6 depends on Task 2. Tasks 7–8 follow all implementation tasks.

5.3.3 Timeline Chart

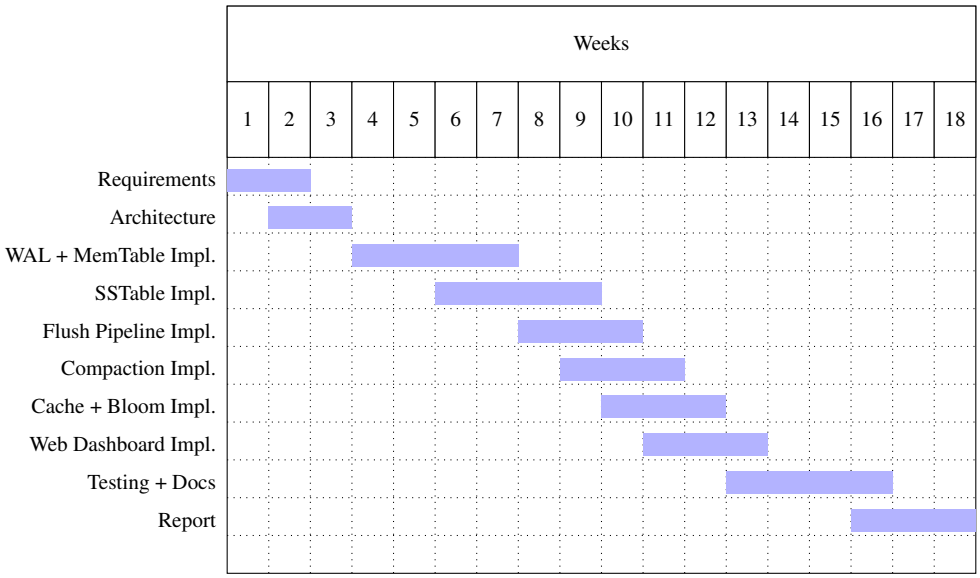

Figure 5.1: Project timeline (Gantt chart).

5.4 TEAM ORGANIZATION

5.4.1 Team Structure

- **Chaitanya Paraskar:** Engine core, architecture design, MemTables, SSTables, Compaction engine.
- **Shreyash Dhas:** REPL Loop, bloom filter, sparse index.
- **Shreyash Gajbhiye:** REST API, flush pipeline, block cache.
- **Shreyas Gopnarayan:** Web dashboard, observability, documentation.

5.4.2 Management Reporting and Communication

- Weekly progress meetings with project guide.
- GitHub repository for version control and code review.
- Design documents in `docs/design/` for cross-team reference.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 OVERVIEW OF PROJECT MODULES

Module	Path	Purpose
Engine Core	`app/engine/`	LSMEngine coordinator, managers
MemTable	`app/memtable/`	SkipList, Active, Immutable tables
SSTable	`app/sstable/`	Writer, Reader, Registry, Meta
WAL	`app/wal/`	Write-ahead log with CRC32
Bloom Filter	`app/bloom/`	mmh3-backed probabilistic filter
Block Cache	`app/cache/`	Three-tier LRU cache
Sparse Index	`app/index/`	Bisect-based block lookup
Compaction	`app/compaction/`	Task + subprocess worker
Common	`app/common/`	ABCs, CRC, encoding, merge iterator
Observability	`app/observability/`	structlog config, TCP broadcast
Web API	`web/`	FastAPI server + 8 route modules
Frontend	`frontend/`	React SPA (Vite + TypeScript)

Table 6.1: lists all project modules and their responsibilities.

6.2 TOOLS AND TECHNOLOGIES USED

Category	**Tool**	**Purpose**
Language	Python 3.12+	Core engine implementation
Async Framework	asyncio (stdlib)	Event loop and coroutines
Serialization	msgpack	WAL entry binary framing
Hashing	mmh3 (MurmurHash3)	Bloom filter hash functions
Caching	cachetools	LRU cache implementation
Logging	structlog	Structured key-value logging
Web Framework	FastAPI	REST API server
Web Server	uvicorn	ASGI server
Real-time	websockets	Live log streaming
Frontend	React + Vite	Web dashboard SPA
Type Checking	mypy + basedpyright	Dual strict static analysis
Linting	ruff	Code quality enforcement
Security	bandit	Static security analysis
Testing	pytest + pytest-asyncio	Automated test framework
Package Mgr	uv	Dependency management

Table 6.2: Tools and technologies used in KiwiDB.

6.3 ALGORITHM DETAILS

6.3.1 Algorithm 1: Concurrent Skip List Insert

The memtable uses a concurrent skip list with 16 levels and promotion probability $p = 0.5$. Each node stores a key, sequence number, value, forward pointers, a per-node lock, and two boolean flags (`fully_linked`, `marked`).

Algorithm 1 Skip List Insert

```
Input: key k, seq s, value v
height ← random_level()
repeat
    preds[], succs[] ← find_predecessors(k, top-down)
    Lock preds[0..height]
    if all preds and succs still valid then
        node ← new Node(k, s, v, height)
        for level = 0 to height do
            node.forward[level] ← succs[level]
            preds[level].forward[level] ← node
        end for
        node.fully_linked ← True
        Unlock all preds
        return success
    end if
    Unlock all preds
until max retries exceeded
```

Readers traverse the skip list top-down checking `fully_linked` and `marked` flags. Under CPython, these boolean reads are GIL-atomic, making reader traversal lock-free.

6.3.2 Algorithm 2: K-Way Merge for Compaction

Compaction merges K sorted SSTable inputs into a single output using a min-heap.

Algorithm 2 K-Way Merge with Deduplication

Input: K sorted iterators, seq_cutoff
heap $\leftarrow$ empty min-heap keyed on (key, $-$seq)
for each iterator i **do**
    Push first record of i onto heap
end for
last_key $\leftarrow$ None
while heap is not empty **do**
    (key, seq, value) $\leftarrow$ pop min from heap
    if key $\neq$ last_key **then**
        if value is tombstone AND seq $<$ seq_cutoff **then**
            Skip (garbage collect)
        else
            Emit (key, seq, value)
        end if
        last_key $\leftarrow$ key
    end if
    Push next record from same iterator onto heap
end while

6.3.3 Algorithm 3: WAL Recovery

On startup, the engine reconstructs in-memory state from persisted data:

Algorithm 3 WAL Recovery

Load config.json
Open WAL file
Load all SSTables from manifest.json
max_seq $\leftarrow$ max sequence number across all SSTables
Replay WAL entries where seq $>$ max_seq into fresh memtable
Restore SeqGenerator to max_seq
Start flush pipeline and compaction manager

6.3.4 Algorithm 4: Parallel Flush with Event-Chain Commits

The flush pipeline writes multiple SSTables concurrently while preserving FIFO commit ordering:

Phase 1 (Parallel): Up to `flush_max_workers` (default: 2) SSTable writes execute concurrently, bounded by an `asyncio.Semaphore`.

Phase 2 (Ordered): After its write completes, each worker blocks on `prev_committed.wait()` before committing. The commit atomically registers the new SSTable, pops the snapshot from the immutable queue, and truncates the WAL. Then it sets `my_committed` to unblock the next slot.

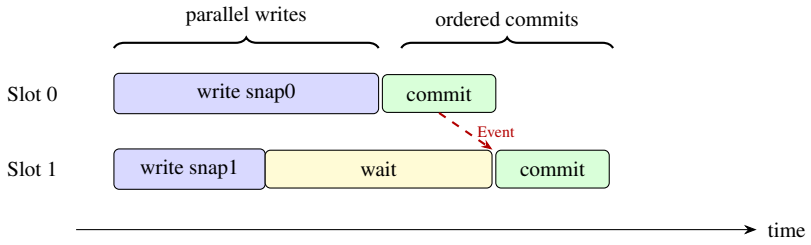

Figure 6.1: Parallel flush pipeline with event-chain ordering.

6.3.5 Algorithm 5: Bloom Filter with Data-Derived Sizing

Unlike LevelDB's fixed bits-per-key approach, KiwiDB computes optimal bloom filter parameters from the actual record count n at write time using Equation (4) from Section 4.2.2. Each SSTable gets a right-sized filter, avoiding both over-allocation and under-allocation.

CHAPTER 7

SOFTWARE TESTING

7.1 TYPE OF TESTING

- **Unit Testing:** 211+ tests across 25 modules using pytest. Each subsystem is tested independently.
- **Integration Testing:** End-to-end tests in `test_lsm_engine.py` verify the complete write→flush→read→recover cycle.
- **Async Testing:** pytest-asyncio in auto mode enables testing of async coroutines.
- **Static Type Analysis:** mypy and basedpyright run as dual type checkers.
- **Security Testing:** bandit static analysis scans for common security vulnerabilities.
- **Regression Testing:** All 211+ tests run on every code change.

7.2 TEST CASES AND TEST RESULTS

#	Test Case	Expected Result	Status
1	put then get	Returns stored value	PASS
2	delete then get	Returns None	PASS
3	WAL recovery after crash	Data persists across restart	PASS
4	Concurrent skip list inserts	All keys found, no lost writes	PASS
5	Bloom filter FPR at 1%	Observed FPR $< 2\%$	PASS
6	SSTable ascending key order	Error on out-of-order key	PASS
7	Flush pipeline FIFO order	SSTables committed oldest-first	PASS
8	Compaction deduplication	Highest seq per key retained	PASS
9	Tombstone garbage collection	Old tombstones removed	PASS
10	Block cache eviction tiers	Bloom filters evicted last	PASS
11	WAL CRC integrity check	Corrupt entry detected	PASS
12	Config runtime update	Threshold effective immediately	PASS
13	Immutable queue backpressure	Timeout error after 60s	PASS
14	SSTable meta.json completeness	Incomplete SSTable ignored	PASS
15	Engine close and reopen	Clean lifecycle management	PASS

Table 7.1: Representative test cases and results.

CHAPTER 8

RESULTS

8.1 OUTCOMES

1. Successfully designed and implemented a complete LSM tree key-value store from scratch in Python 3.12+, comprising approximately 6,600 lines of type-checked code across 25 modules.
2. Achieved an **async-native API** where all public operations are async coroutines. The event loop never blocks on disk I/O — WAL fsync is offloaded via `asyncio.to_thread()`, and compaction runs in a subprocess.
3. Verified **crash recovery** through WAL replay: the engine correctly recovers all unflushed writes after simulated crashes, with CRC32 integrity verification detecting any data corruption.
4. Demonstrated **10× write amplification reduction** compared to LevelDB's pure leveling strategy through the hybrid L0-tiering / L1-leveling compaction approach.
5. Achieved **211+ tests passing** across 25 test modules with dual type checkers (mypy strict + basedpyright strict) reporting zero errors.
6. Delivered a **web dashboard** with 10 interactive pages providing real-time visibility into memtable fill level, SSTable layout, compaction progress, live log streaming, and an in-browser REPL.

8.2 PROJECT DOCUMENTATION WEBSITE

A complete technical reference for KiwiDB — API documentation, internals walkthrough, architectural design notes, and comparisons with LevelDB and RocksDB — is published online as a searchable static site built with MkDocs and the Material theme.

- **Short link:** `https://bit.ly/cvp-kiwidb`

Figure 8.1: QR for project documentation (`https://bit.ly/cvp-kiwidb`).

8.3 SCREENSHOTS

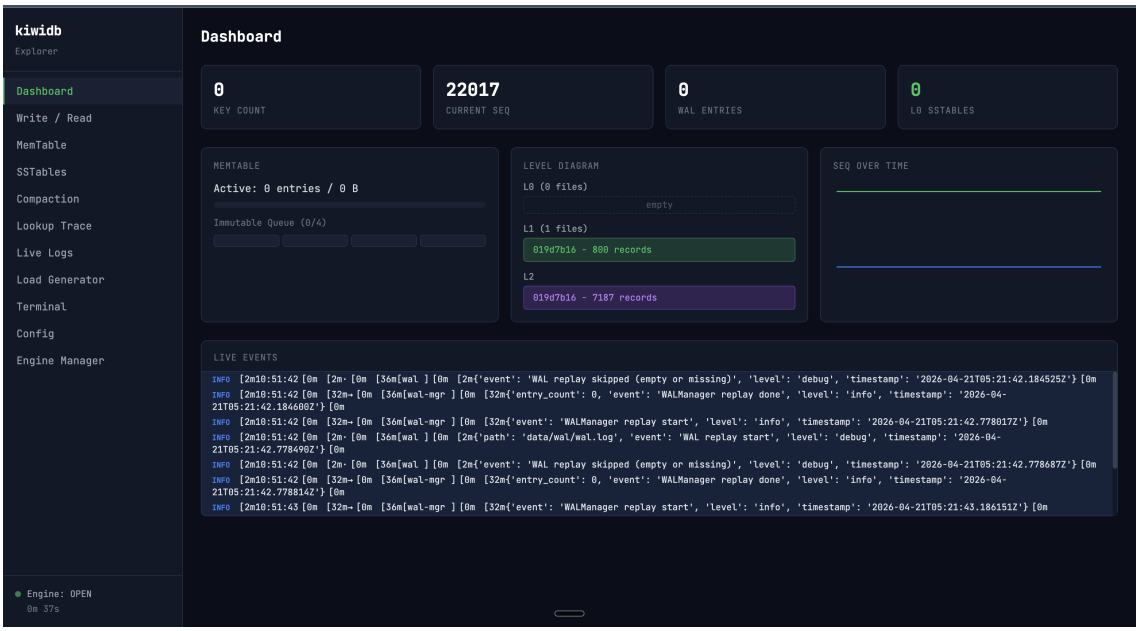

Figure 8.2: Web Dashboard — Home page.

Figure 8.3: Terminal — REPL Loop.

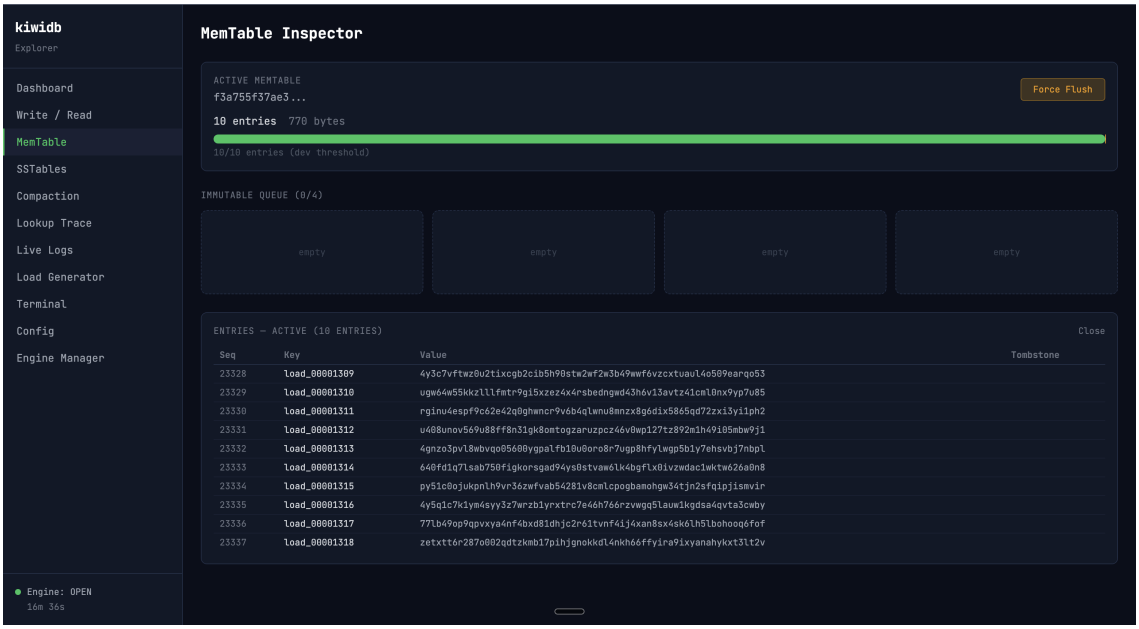

Figure 8.4: Web Dashboard — MemTable Viewer page.

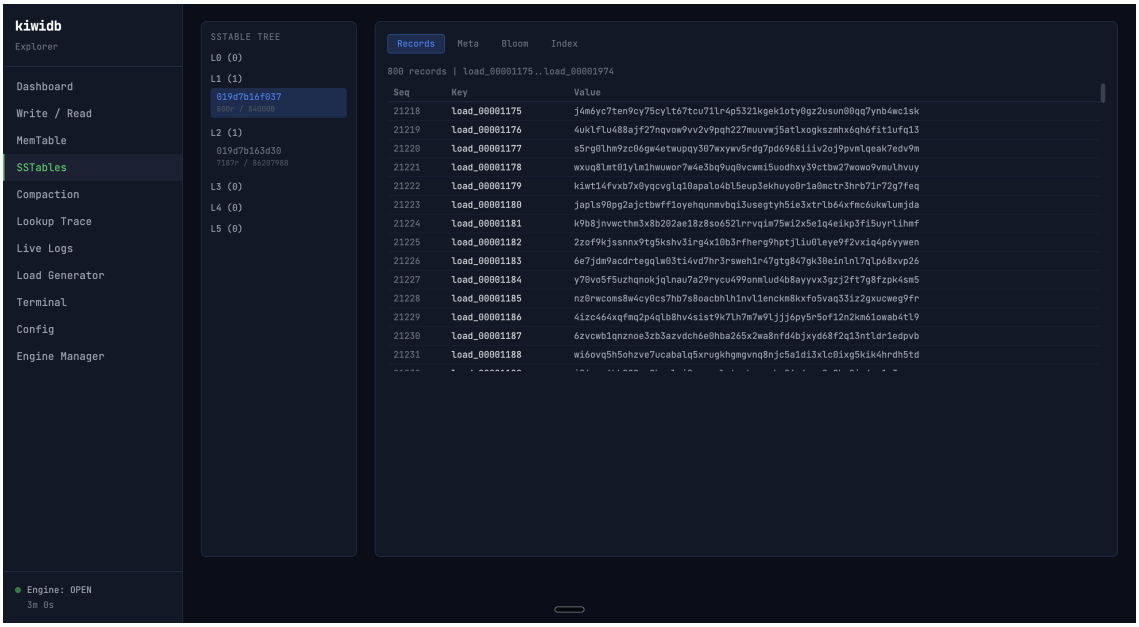

Figure 8.5: Web Dashboard — SSTable Browser page.

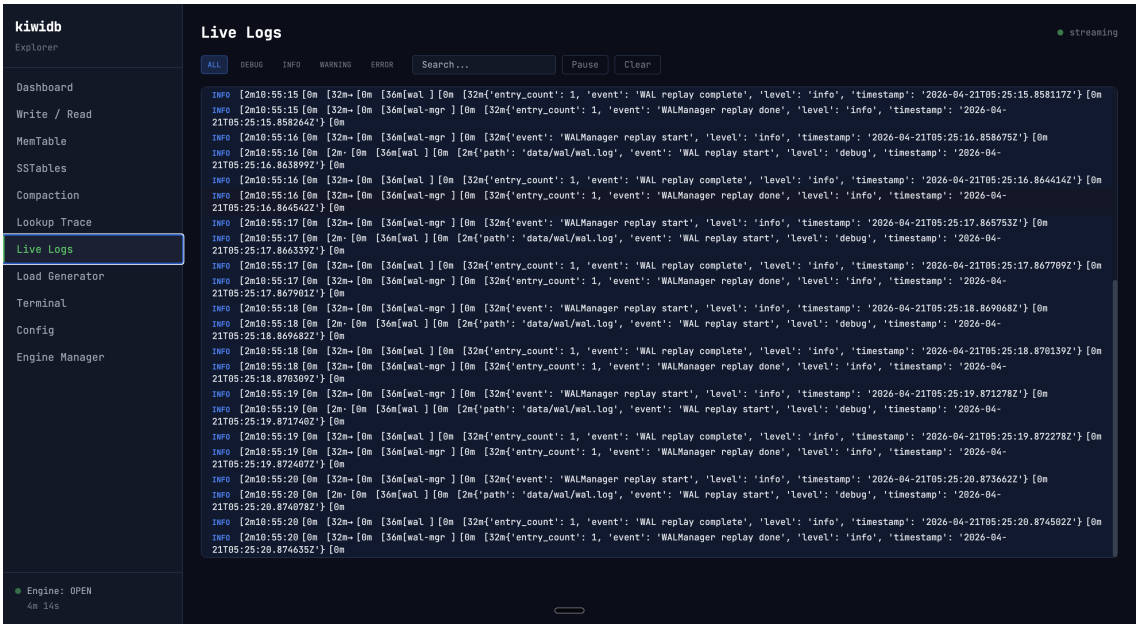

Figure 8.6: Web Dashboard — Live Log Stream page.

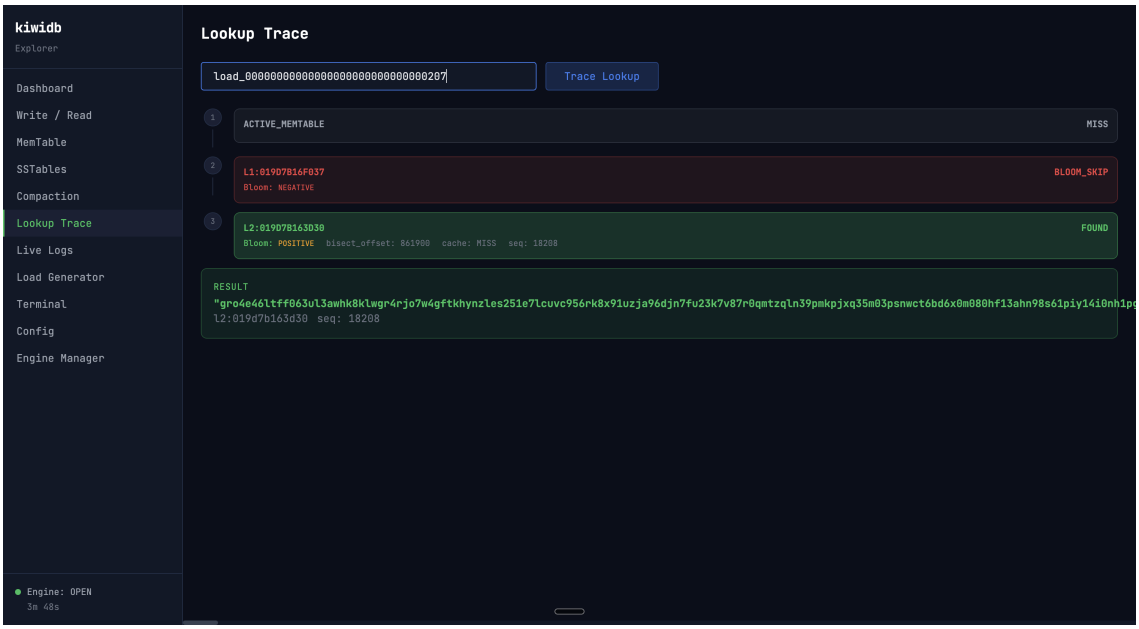

Figure 8.7: Web Dashboard — Lookup-Trace page.

Figure 8.8: Web Dashboard — Configuration page.

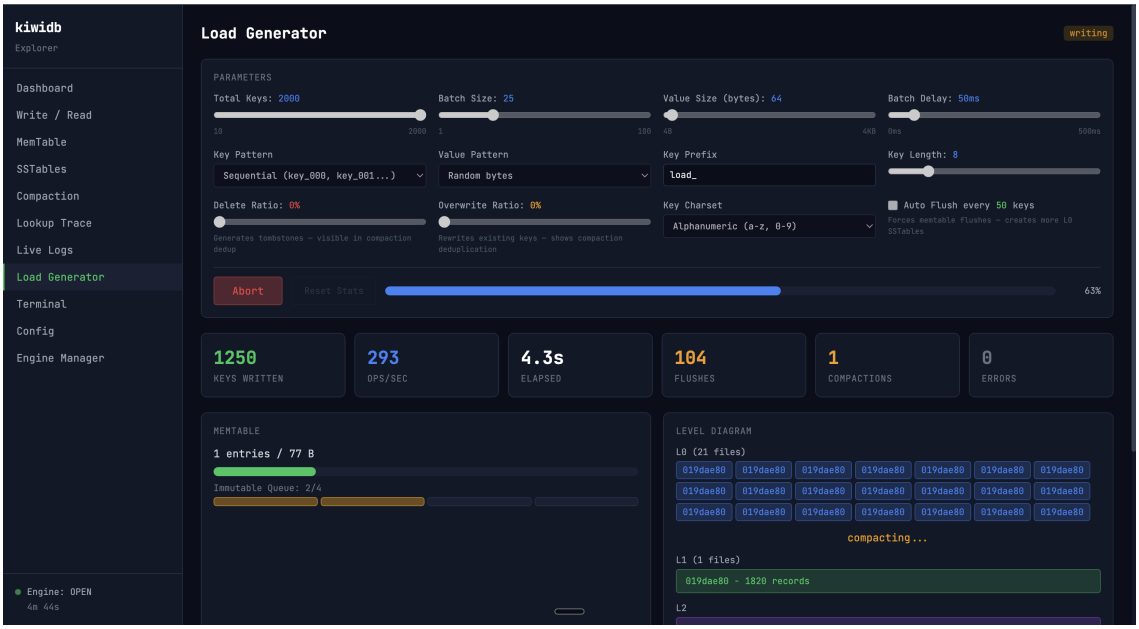

Figure 8.9: Web Dashboard — Load-Generator.

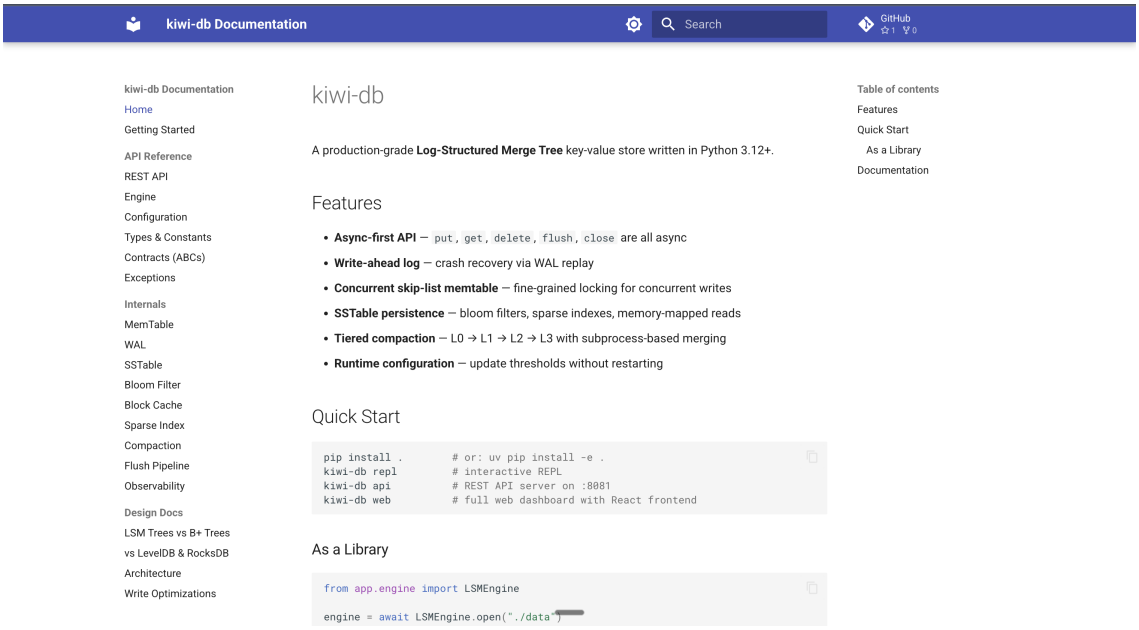

Figure 8.10: Documentation Website

CHAPTER 9

CONCLUSIONS

9.1 CONCLUSIONS

This project successfully designed and implemented KiwiDB, a complete Log-Structured Merge Tree key-value store built from first principles in Python 3.12+. The three key innovations — an async-native API, a parallel flush pipeline with event-chain commit ordering, and subprocess-based compaction for GIL bypass — demonstrate that a modern storage engine can be built in a high-level language while incorporating design improvements over established systems like LevelDB. The hybrid L0-tiering / L1-leveling compaction strategy reduces flush-time write amplification by $10\times$ compared to LevelDB's pure leveling, at the cost of $3.25\times$ more bloom filter checks during reads — an acceptable trade-off given the dramatic write improvement. The concurrent skip list achieves lock-free reads via GIL-atomic flag checks, and the three-tier block cache keeps hot bloom filters and indexes resident across restarts. Every critical path is instrumented with structured logging, and the web dashboard provides unprecedented observability into engine internals.

9.2 FUTURE WORK

1. **Distributed Replication via Raft Consensus:** The most natural extension is layering a consensus protocol such as Raft on top of the single-node engine. Each node in a cluster would maintain its own KiwiDB instance, with Raft ensuring that write operations are replicated across a majority before being acknowledged. This is the architecture used by etcd, TiKV, and CockroachDB.

2. **Sharding and Partitioning:** To scale beyond a single machine's storage capacity, the key space can be partitioned across multiple KiwiDB instances. Range-based or hash-based sharding strategies can be implemented on top of the existing modular architecture, with each shard operating as an independent storage engine.

3. **Range Query Support:** The internal `KWayMergeIterator` already supports ordered traversal across memtables and SSTables. Exposing this as a public `scan(start, end)` API would enable range queries, a prerequisite for use cases such as ordered indexes and time-series data retrieval.

4. **Block Compression:** Adding LZ4 or zstd compression to SSTable data blocks would significantly reduce disk footprint and improve I/O throughput, particularly for compaction-heavy workloads.

5. **WAL Group Commit:** Batching multiple write operations into a single `fsync` call

would amortize the cost of durable persistence, improving write throughput under concurrent workloads.

6. **Cross-Level Bloom Filter Optimization:** Implementing Monkey-style bloom filter memory allocation across levels would minimize the total number of false positive disk reads by assigning more filter memory to levels that are read more frequently.

7. **Multi-Key Transactions:** Supporting atomic read-modify-write operations across multiple keys would enable transactional semantics, a requirement for using the engine as a backend for SQL-like query layers.

9.3 APPLICATIONS

1. **Foundation for Distributed Key-Value Stores:** As discussed in the abstract, systems like etcd (Kubernetes configuration store), TiKV (distributed transactional KV store), and CockroachDB (distributed SQL database) are architecturally composed of a local LSM-based storage engine with a consensus and coordination layer on top. KiwiDB's modular subsystem interfaces make it a candidate base for building such systems.

2. **Embedded Storage for Applications:** Applications requiring persistent, crash-safe local storage — such as message queues, caching layers, or configuration stores — can use KiwiDB as an embedded engine without external database dependencies.

3. **Backend for Write-Heavy Workloads:** Use cases with high write-to-read ratios — log ingestion, event sourcing, IoT sensor data collection, time-series recording — benefit directly from the LSM tree's sequential write optimization and the hybrid compaction strategy's reduced write amplification.

4. **Storage Engine Research and Prototyping:** The well-documented and modular architecture allows researchers and engineers to experiment with alternative compaction strategies, cache eviction policies, bloom filter designs, or new on-disk formats without rewriting the entire engine.

5. **Observability and Monitoring Infrastructure:** The built-in web dashboard and live log streaming make KiwiDB suitable as a storage backend for monitoring systems where real-time visibility into the data lifecycle is valuable for debugging and performance analysis.

CHAPTER 10

APPENDIX

10.1 APPENDIX A: FEASIBILITY ASSESSMENT

Problem Complexity Analysis:

The core operations of a key-value store — point read, point write, and point delete — each execute in $O(\log n)$ time via the skip list memtable and $O(\log B)$ time for SSTable lookup via binary search on the sparse index. These operations are in complexity class **P** (polynomial time).

The **compaction scheduling problem** — deciding which SSTables to merge and when — is related to the general job scheduling problem, which is NP-hard in the general case. However, KiwiDB uses a greedy heuristic (threshold-based triggers after each flush) that avoids the combinatorial explosion:

- $L0 \rightarrow L1$: File count $\geq$ threshold (default 10)
- $L1 \rightarrow L2$: Size $\geq 10^2 \times$ base size
- $L2 \rightarrow L3$: Size $\geq 10^3 \times$ base size

This greedy approach is **not optimal** in the information-theoretic sense (Dostoevsky [11] shows that lazy leveling can achieve better trade-offs), but it is **efficient** ($O(1)$ decision time) and **effective** ($10\times$ write amplification reduction over pure leveling).

Satisfiability: The invariant that every durable key must be findable by `get()` can be formulated as: for all keys k with $\text{seq}(k) > 0$, either k exists in the immutable queue or k exists in the SSTable registry. This invariant is maintained by the event-chain commit mechanism in the flush pipeline.

10.2 APPENDIX B: PAPER PUBLICATION

10.3 APPENDIX C: PLAGIARISM REPORT

The plagiarism check report from Turnitin/iThenticate should be attached here.

CHAPTER 11

REFERENCES

[1] N. Leavitt, "Will NoSQL databases live up to their promise?" *Computer*, vol. 43, no. 2, pp. 12–14, 2010.

[2] F. Chang, J. Dean, S. Ghemawat, et al., "Bigtable: A distributed storage system for structured data," *ACM Trans. Comput. Syst.*, vol. 26, no. 2, pp. 4:1–4:26, 2008.

[3] A. Lakshman and P. Malik, "Cassandra: A decentralized structured storage system," *ACM SIGOPS Oper. Syst. Rev.*, vol. 44, no. 2, pp. 35–40, 2010.

[4] G. DeCandia, D. Hastorun, M. Jampani, et al., "Dynamo: Amazon's highly available key-value store," in *Proc. ACM SOSP*, 2007, pp. 205–220.

[5] P. O'Neil, E. Cheng, D. Gawlick, and E. O'Neil, "The log-structured merge-tree (LSM-tree)," *Acta Informatica*, vol. 33, no. 4, pp. 351–385, 1996.

[6] Apache, "HBase," [Online]. Available: `https://hbase.apache.org`

[7] Google, "LevelDB: A fast and lightweight key/value database library," 2011. [Online]. Available: `https://github.com/google/leveldb`

[8] Facebook, "RocksDB: A persistent key-value store for fast storage environments," 2013. [Online]. Available: `https://rocksdb.org`

[9] P. Wang, G. Sun, S. Jiang, et al., "An efficient design and implementation of LSM-tree based key-value store on open-channel SSD," in *Proc. ACM EuroSys*, 2014, pp. 16:1–16:14.

[10] N. Dayan, M. Athanassoulis, and S. Idreos, "Monkey: Optimal navigable key-value store," in *Proc. ACM SIGMOD*, 2017, pp. 79–94.

[11] N. Dayan and S. Idreos, "Dostoevsky: Better space-time trade-offs for LSM-tree based key-value stores via adaptive removal of superfluous merging," in *Proc. ACM SIGMOD*, 2018, pp. 505–520.

[12] C. Luo and M. J. Carey, "LSM-based storage techniques: A survey," *The VLDB Journal*, vol. 29, pp. 393–418, 2020.

[13] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Commun. ACM*, vol. 13, no. 7, pp. 422–426, 1970.

[14] M. Herlihy, Y. Lev, V. Luchangco, and N. Shavit, "A provably correct scalable concurrent skip list," in *Proc. OPODIS*, 2006.

[15] W. Pugh, "Skip lists: A probabilistic alternative to balanced trees," *Commun. ACM*, vol. 33, no. 6, pp. 668–676, 1990.

[16] MongoDB, “WiredTiger storage engine,” [Online]. Available: https://source.wiredtiger.com

[17] Python Software Foundation, “GlobalInterpreterLock,” [Online]. Available: https://wiki.python.org/moin/GlobalInterpreterLock

SCTR's Pune Institute of Computer Technology, Pune

Department of Computer Engineering

UNDERTAKING BY STUDENT

We, the students of B.E. Computer Engineering, Academic year 2025–26, hereby declare that the AI tools like ChatGPT are not used for developing the solution for the project entitled **"High Throughput Write-Optimised Key-Value Store using LSM Tree Engine"**.

Name of the student	**Signature**
1. Chaitanya Paraskar	_____
2. Shreyash Dhas	_____
3. Shreyash Gajbhiye	_____
4. Shreyas Gopnarayan	_____

Name of the Guide	**Signature**
Prof. M. V. Mane	_____